

WESAD - Pipeline BVP da polso su S2

Documento riassuntivo del lavoro svolto: setup Python, lettura di S2.pkl, estrazione del segnale BVP, creazione di finestre temporali, feature extraction, label binarie e generazione del CSV.

Obiettivo pratico

Abbiamo costruito la prima pipeline funzionante del progetto WESAD partendo da un solo soggetto, un solo segnale e un solo task. Il soggetto usato e' S2, il segnale e' BVP dal polso, e il task e' classificazione binaria stress vs non-stress.

Idea chiave: non si addestra il modello direttamente sui milioni di valori grezzi del segnale. Prima si trasforma il segnale temporale in una tabella di feature, dove ogni riga rappresenta una finestra temporale di 60 secondi.

Indice

1. Struttura del progetto in VS Code
2. Lettura corretta di S2.pkl
3. Che cosa contiene il file S2.pkl
4. Perche' il BVP non ha la stessa lunghezza delle label
5. Windowing: da segnale grezzo a righe di tabella
6. Label originali e label binarie
7. Regola di purezza al 70%
8. Feature estratte dal BVP
9. Output finale: bvp_features_s2.csv
10. Cosa NON dare al modello
11. Codice finale usato
12. Prossimi passi

1. Struttura del progetto in VS Code

Abbiamo creato una cartella di progetto chiamata WESAD_Project e l'abbiamo aperta in Visual Studio Code. Dentro questa cartella sono presenti l'ambiente virtuale Python, la cartella del soggetto S2, gli script Python e il CSV finale.

```
WESAD_Project/
├── .venv/
├── S2/
│   ├── S2.pkl
│   ├── S2_E4_Data.zip
│   ├── S2_quest.csv
│   ├── S2_readme.txt
│   └── S2_respiban.txt
├── read_s2.py
├── create_bvp_features_s2.py
└── bvp_features_s2.csv
```

Abbiamo anche risolto il problema iniziale di Python e pip installando Python, creando un ambiente virtuale .venv e installando le librerie necessarie: numpy, pandas, scikit-learn e matplotlib.

Comandi principali usati

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install numpy pandas scikit-learn matplotlib
```

In PowerShell e' stato necessario abilitare gli script per attivare l'ambiente virtuale:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

2. Lettura corretta di S2.pkl

Il file S2.pkl non deve essere aperto come testo. Quando lo si apre a mano, si vede una sequenza incomprensibile, per esempio '(dp0 S signal ...', perche' e' un file pickle Python, cioe' un oggetto salvato in formato binario.

Il modo corretto per leggerlo e' usare pickle.load in Python.

```
import pickle
from pathlib import Path

path = Path('S2') / 'S2.pkl'

with open(path, 'rb') as f:
    data = pickle.load(f, encoding='latin1')

print(type(data))
print(data.keys())
```

Risultato ottenuto:

```
TIPO DATA: <class 'dict'>
CHIAVI PRINCIPALI: dict_keys(['signal', 'label', 'subject'])
SOGGETTO: S2
```

3. Che cosa contiene il file S2.pkl

S2.pkl contiene tutti i segnali sincronizzati del soggetto S2, divisi tra dispositivo sul petto e dispositivo al polso, piu' le label del protocollo sperimentale.

Chiave	Contenuto	Uso nel progetto
subject	ID del soggetto, in questo caso S2	Serve per sapere da quale soggetto arrivano le righe

signal	Dizionario con segnali chest e wrist	Da qui prendiamo il BVP
label	Etichetta temporale della condizione sperimentale	Serve per assegnare stress/non-stress alle finestre

Segnali disponibili

Dispositivo	Segnali	Frequenze principali
Chest - RespiBAN	ACC, ECG, EMG, EDA, Temp, Resp	700 Hz
Wrist - Empatica E4	ACC, BVP, EDA, TEMP	ACC 32 Hz, BVP 64 Hz, EDA/TEMP 4 Hz

Output letto da S2.pkl

```

SEGNALI CHEST:
ACC (4255300, 3)
ECG (4255300, 1)
EMG (4255300, 1)
EDA (4255300, 1)
Temp (4255300, 1)
Resp (4255300, 1)

SEGNALI WRIST:
ACC (194528, 3)
BVP (389056, 1)
EDA (24316, 1)
TEMP (24316, 1)

LABEL:
Shape: (4255300, )

```

4. Perché il BVP non ha la stessa lunghezza delle label

Il BVP del polso è campionato a 64 Hz: significa che per ogni secondo abbiamo 64 valori BVP. Le label invece sono campionate a 700 Hz, quindi per ogni secondo abbiamo 700 valori di label. Per questo BVP e label non hanno la stessa lunghezza.

Elemento	Frequenza	Significato
BVP wrist	64 Hz	64 valori al secondo
Label	700 Hz	700 valori al secondo
Finestra 60s BVP	64 x 60	3840 valori BVP
Finestra 60s label	700 x 60	42000 valori label

Quindi non si può collegare `bvp[i]` direttamente a `labels[i]`. Bisogna ragionare per tempo: stessa finestra temporale, ma numero diverso di campioni.

5. Windowing: da segnale grezzo a righe di tabella

Il segnale BVP di S2 contiene 389.056 valori. Il modello però non deve ricevere questi valori grezzi. Abbiamo deciso di tagliare il segnale in finestre temporali da 60 secondi con step di 30 secondi.

Window = 60 secondi significa che ogni riga del CSV rappresenta un pezzo di BVP lungo 60 secondi. Step = 30 secondi significa che la finestra successiva parte 30 secondi dopo, quindi le finestre si sovrappongono al 50%.

```

Finestra 1: 0s - 60s
Finestra 2: 30s - 90s
Finestra 3: 60s - 120s
Finestra 4: 90s - 150s

```

Ogni finestra da 60 secondi contiene 3840 valori BVP. Da questi 3840 valori calcoliamo poche feature statistiche. Così una sequenza lunga diventa una riga di tabella.

Sintesi del passaggio

```
389.056 valori BVP grezzi
→ finestre da 60 secondi
→ 3840 valori BVP per finestra
→ calcolo feature
→ 1 riga del CSV per ogni finestra utile
```

6. Label originali e label binarie

Nel dataset WESAD le label originali identificano la condizione sperimentale. Per il nostro primo progetto non usiamo tutte le label: teniamo solo baseline, stress e amusement. Le altre vengono scartate.

Label originale	Significato	Decisione nel progetto
0	Not defined / transient	Scartata
1	Baseline	Tenuta - diventa non-stress
2	Stress	Tenuta - diventa stress
3	Amusement	Tenuta - diventa non-stress
4	Meditation	Scartata nella prima pipeline
5/6/7	Da ignorare	Scartate

La classificazione che vogliamo fare è binaria: stress vs non-stress. Per questo trasformiamo le label originali così:

```
original_label 1 = baseline → label 0 = non-stress
original_label 2 = stress   → label 1 = stress
original_label 3 = amusement → label 0 = non-stress
```

Nel CSV finale teniamo entrambe le colonne: original_label per sapere la label WESAD originale, e label per il task binario.

7. Regola di purezza al 70%

Abbiamo aggiunto una regola per evitare finestre troppo miste, cioè finestre in cui le label interne sono molto diverse tra loro. Per ogni finestra guardiamo qual è la label prevalente e calcoliamo la sua percentuale.

```
label_purity = campioni della label prevalente / campioni totali nella finestra
```

La regola scelta è: tenere la finestra solo se almeno il 70% dei campioni appartiene alla label prevalente.

Esempio finestra	Purezza	Decisione
90% baseline, 10% altro	0.90	Tengo la finestra
72% stress, 28% altro	0.72	Tengo la finestra
60% baseline, 40% stress	0.60	Scarto la finestra

Questa regola usa le label vere del dataset, quindi è una strategia di preprocessing supervisionato. Va bene per costruire un dataset pulito in un benchmark offline, ma in un sistema reale non si potrebbe usare nello stesso modo, perché la label vera non sarebbe nota prima della predizione.

8. Feature estratte dal BVP

Per ogni finestra da 60 secondi vengono calcolate feature statistiche sul segnale BVP. Nella versione corrente del codice sono presenti anche median ed energy. Se si vuole una baseline ancora piu' semplice, si possono tenere solo mean, std, min, max e range.

Feature	Significato	Nota
bvp_mean	Media dei valori BVP nella finestra	Riassume il livello medio
bvp_std	Deviazione standard	Misura quanto varia il segnale
bvp_min	Valore minimo	Estremo inferiore nella finestra
bvp_max	Valore massimo	Estremo superiore nella finestra
bvp_range	max - min	Ampiezza totale del segnale
bvp_median	Mediana	Opzionale, simile alla media ma meno sensibile agli estremi
bvp_energy	Media dei quadrati	Opzionale; se usata, meglio come media e non come somma

Esempio mentale

```
3840 valori BVP in una finestra da 60s
→ bvp_mean
→ bvp_std
→ bvp_min
→ bvp_max
→ bvp_range
→ bvp_median
→ bvp_energy
→ label stress/non-stress
```

9. Output finale: bvp_features_s2.csv

Lo script `create_bvp_features_s2.py` genera il file `bvp_features_s2.csv`. Questo file e' il primo vero dataset tabellare pronto per il machine learning. Ogni riga rappresenta una finestra temporale del segnale BVP.

Colonna	Descrizione
subject	Soggetto da cui arriva la finestra, per ora S2
start_sec	Secondo di inizio della finestra
end_sec	Secondo di fine della finestra
window_sec	Durata della finestra: 60
step_sec	Step usato: 30
bvp_*	Feature calcolate sul segnale BVP
original_label	Label originale WESAD: 1, 2 o 3
label	Label binaria: 0 non-stress, 1 stress
label_purity	Percentuale della label prevalente nella finestra

Nel primo test senza regola di purezza erano state create 71 righe. Con la regola di purezza al 70%, il numero atteso per la finestra da 60s e' circa 67 righe per S2, perche' alcune finestre ambigue vengono scartate.

10. Cosa NON dare al modello

Questo e' uno dei punti piu' importanti. Il modello non deve ricevere la label come input. La label serve solo per addestrare e valutare.

Elemento	Ruolo
X	Feature del segnale: bvp_mean, bvp_std, bvp_min, bvp_max, bvp_range, ecc.
y	Label corretta da predire: 0 non-stress oppure 1 stress
original_label	Metadato utile per analisi, ma non va dato al modello
label_purity	Metadato utile per controllare la pulizia, ma non va dato al modello nella prima baseline

Corretto

```
X = df[['bvp_mean', 'bvp_std', 'bvp_min', 'bvp_max', 'bvp_range']]
y = df['label']
```

Sbagliato

```
X = df[['bvp_mean', 'bvp_std', 'bvp_min', 'bvp_max', 'bvp_range', 'label']] # NO
```

Se si mette label dentro X, il modello riceve gia' la risposta. Questo si chiama data leakage e rende la valutazione falsa.

11. Codice finale usato

Questo e' il codice aggiornato con finestra da 60 secondi, step da 30 secondi e soglia di purezza al 70%.

```
import pickle
import numpy as np
import pandas as pd
from pathlib import Path

# =====
# 1. Caricamento dati S2
# =====

path = Path("S2") / "S2.pkl"

with open(path, "rb") as f:
    data = pickle.load(f, encoding="latin1")

bvp = data["signal"]["wrist"]["BVP"].ravel()
labels = data["label"]

# Frequenze di campionamento
fs_bvp = 64      # BVP wrist = 64 Hz
fs_label = 700   # label = 700 Hz

# Parametri finestra
window_sec = 60
step_sec = 30

# Soglia di purezza della label
purity_threshold = 0.70

total_seconds = len(labels) // fs_label

rows = []

# =====
# 2. Creazione finestre
# =====

for start_sec in range(0, total_seconds - window_sec, step_sec):
    end_sec = start_sec + window_sec
```

```

# Finestra BVP
bvp_start = start_sec * fs_bvp
bvp_end = end_sec * fs_bvp
bvp_window = bvp[bvp_start:bvp_end]

# Finestra label
label_start = start_sec * fs_label
label_end = end_sec * fs_label
label_window = labels[label_start:label_end]

if len(bvp_window) == 0 or len(label_window) == 0:
    continue

# Label prevalente nella finestra
values, counts = np.unique(label_window, return_counts=True)
majority_index = np.argmax(counts)
majority_label = values[majority_index]
majority_ratio = counts[majority_index] / len(label_window)

# Tengo solo label utili: 1 baseline, 2 stress, 3 amusement
if majority_label not in [1, 2, 3]:
    continue

# Scarto finestre troppo miste
if majority_ratio < purity_threshold:
    continue

# Task binario: stress = 1, non-stress = 0
binary_label = 1 if majority_label == 2 else 0

# =====
# 3. Feature extraction
# =====

row = {
    "subject": data["subject"],
    "start_sec": start_sec,
    "end_sec": end_sec,
    "window_sec": window_sec,
    "step_sec": step_sec,

    "bvp_mean": np.mean(bvp_window),
    "bvp_std": np.std(bvp_window),
    "bvp_min": np.min(bvp_window),
    "bvp_max": np.max(bvp_window),
    "bvp_range": np.max(bvp_window) - np.min(bvp_window),
    "bvp_median": np.median(bvp_window),
    "bvp_energy": np.mean(bvp_window ** 2),

    "original_label": majority_label,
    "label": binary_label,
    "label_purity": majority_ratio
}

rows.append(row)

# =====
# 4. Creazione DataFrame
# =====

df = pd.DataFrame(rows)

print(df.head())
print("\nShape tabella:", df.shape)

print("\nDistribuzione label binarie:")
print(df["label"].value_counts())

print("\nDistribuzione label originali:")
print(df["original_label"].value_counts())

print("\nPurezza minima delle finestre:")
print(df["label_purity"].min())

```

```
# Salvataggio CSV
df.to_csv("bvp_features_s2.csv", index=False)

print("\nFile salvato: bvp_features_s2.csv")
```

12. Prossimi passi

Ora la pipeline funziona su un singolo soggetto. Il prossimo obiettivo e' estenderla a tutti i soggetti, uno alla volta, senza caricare tutti i file .pkl insieme in memoria.

Step	Cosa fare	Risultato atteso
1	Ripetere la feature extraction per tutti i soggetti	bvp_features_all_subjects_60s.csv
2	Creare X e y	X = feature BVP, y = stress/non-stress
3	Normalizzare le feature	StandardScaler, fit solo sul training set
4	Addestrare k-NN	kNN con k = 9, coerente con il paper
5	Validare per soggetto	Leave-One-Subject-Out
6	Calcolare metriche	Accuracy, precision, recall, F1-score, confusion matrix
7	Confrontare con il benchmark	BVP + kNN nella Tabella 4 del paper WESAD

Regola pratica finale

Non stiamo ancora cercando il modello migliore. Stiamo costruendo una pipeline corretta. Prima facciamo funzionare BVP + kNN. Poi replichiamo la stessa logica su EDA, ECG, EMG, RESP e sulle combinazioni di segnali.

Frase da usare nella presentazione

Abbiamo implementato una pipeline incrementale su WESAD: lettura del file .pkl, estrazione del segnale BVP da polso, segmentazione in finestre da 60 secondi, assegnazione della label binaria stress/non-stress, filtro di purezza al 70%, feature extraction statistica e generazione del dataset tabellare per il successivo addestramento di un modello k-NN.

Fonti usate nel progetto

- WESAD Dataset README: struttura dei file, frequenze dei sensori, significato delle label e formato del file .pkl.
- Paper WESAD: Schmidt et al., Introducing WESAD, a Multimodal Dataset for Wearable Stress and Affect Detection, ICMI 2018.